ChaTSFM: Channel Adapter for Frozen Time Series Foundation Models (Simplified Reproduction)

Model Research Report (Sample spec, PyTorch) — model_type: TimeSeries — based on Li et al., *Channel Adapter for Time Series Foundation Models in Zero-Shot Multivariate Forecasting* (ChaTSFM, ICML 2026)

1. Overview and model card

Time series foundation models (TSFMs) are typically pre-trained channel-independently, so they under-use inter-channel correlations. ChaTSFM is a lightweight plug-and-play adapter (well under 1% extra parameters) that lets a FROZEN backbone exploit multivariate correlations in a zero-shot manner. Three pillars: (1) budgeted synthetic pre-training over diverse inter-channel dependency patterns; (2) data-derived per-channel 'domain descriptors' that condition a similarity metric; (3) gated sparse refinement blending other channels' backbone forecasts into each channel's prediction. Model card:

```
model_name: ChaTSFMModel
model_type: TimeSeries
description: Frozen lag-linear backbone (stand-in TSFM, weights kept in buffers) plus a trainable channel
adapter (metric weights m, gate scalars b, c) that refines the target channel forecast with the top-k
most similar channels; predicts the next-step value of channel 0.
hyperparameters: n_lags = 12; period = 24; top_k = 2; descriptor_dim = 6
training_hyperparameters: n_epochs = 80; lr = 5e-2; early_stop = 20; batch_size = 64; weight_decay = 0
```

2. Integration contract (mandatory)

The deliverable is ONE Python file named model.py containing exactly one class subclassing torch.nn.Module and the module-level assignment model_cls = <YourClass>. The automated runner applies this contract verbatim:

```
import torch
from model import model_cls
m = model_cls(num_features=10, num_timesteps=4)
for _, param in m.named_parameters():
    param.data.fill_(1.0)
out = m(torch.full((32, 4, 10), 1.0))
assert out.shape == (32, 1) and torch.isfinite(out).all()
```

Consequences: (a) __init__(self, num_features, num_timesteps, **kwargs) with defaults for everything else; (b) forward receives shape (batch_size, num_timesteps, num_features) and returns (batch_size, 1) - do NOT permute the input, timesteps are already axis 1; (c) with every nn.Parameter overwritten by 1.0 and a constant all-ones input the output must stay finite: no division by parameters, no BatchNorm (zero-variance batch gives NaN), prefer LayerNorm with eps or tanh/sigmoid gates; (d) fixed constants belong in register_buffer (buffers are NOT overwritten); (e) no side effects at import time, no try-except, no top-level main; the demo of this report must live inside an if __name__ == '__main__': guard. Only torch and the Python standard library may be imported. model_type: TimeSeries.

3. Model formulation

forward(x) with x of shape (B, T, C); target channel 0; output (B, 1) is the refined next-step forecast of channel 0.

Frozen backbone (a stand-in channel-independent TSFM): per channel, a linear forecast from lags 1..12 plus sin/cos time-of-day features with period 24: f_i = [lags_i, sin_i, cos_i] @ backbone_w. backbone_w lives in a register_buffer (shape (n_lags + 2, 1), initialized to zeros) so the acceptance test's parameter fill does not touch it and the unfitted model outputs zeros. The demo fits backbone_w by closed-form ridge (alpha = 1.0) and writes it into the buffer; after that it is frozen.

Domain descriptors: per channel d_c = [mean, std, trend slope, lag-1 autocorrelation, spectral entropy of the periodogram, dominant-period strength] computed from the window, then z-scored across channels with eps.

Similarity: sim(i, j) = exp(-(d_i - d_j)^T diag(m) (d_i - d_j)) with m = softplus(param_m), param_m an nn.Parameter(6) - the dataset-conditioned metric. Gate: for the target channel take the top k = 2 most similar other channels, normalized

weights w_ij from their similarities; g = sigmoid(b + c * mean_top_k_similarity) with scalars b, c as nn.Parameters. Final y = Linear(1, 1)((1 - g) * f_0 + g * sum_j w_ij f_j), shape (B, 1). Trainable parameters: param_m (6), b, c and the head - a tiny adapter, mirroring the paper.

4. Stability requirements

In the acceptance test all parameters are 1.0: m = softplus(1) > 0 so similarities are valid exp(-nonneg) in (0, 1]; g = sigmoid(1 + 1 * sim) in (0.5, 1); backbone buffer is zero so every f_i = 0 and the output is exactly 0 - finite. Use eps = 1e-6 in every z-score/division. No BatchNorm. Spectral entropy of a flat periodogram must not NaN: normalize the periodogram by its sum + eps before the entropy.

5. Demo: adapter pre-training and zero-shot evaluation (__main__ guard only)

Pre-training corpus inside the guard (torch.manual_seed(42)): S = 6 synthetic multivariate datasets, each C = 6 channels, T_total = 600 steps, covering lead-lag copies, instantaneous scaling, shared seasonality with channel-specific noise, a regime shift in correlation, and fully independent channels. On each: fit the frozen backbone, then optimize ONLY (param_m, b, c, head) with Adam (lr 5e-2, max 80 epochs, batch 64) to minimize the adapter's next-step MSE. Zero-shot evaluation: generate 2 fresh datasets (seed 7) with mixed patterns never used in pre-training; fit fresh frozen backbones; apply the pre-trained adapter; compare MSE on the last 20% of each series: backbone-only vs backbone + adapter. Print per-dataset relative MSE improvement. Success: the adapter improves or matches average MSE on both test datasets.

6. Reference

Dongyuan Li, Renhe Jiang, Shun Zheng, Zheng Dong, Haotian Gao, Ying Zhang, Jiang Bian. Channel Adapter for Time Series Foundation Models in Zero-Shot Multivariate Forecasting. ICML 2026. OpenReview: https://openreview.net/forum?id=OJriSoFuDq . Code: https://github.com/Clearloveyuan/ChaTSFM